

CPRO 2901 A

Paper Party Final Proposal



Paper Party Final Proposal

Paper Party Final Proposal-----	1
Group 1 Contract-----	2
Project Statement-----	4
Problem Definition and Scope-----	4
Stakeholders & Use context-----	5
Functional Requirements-----	6
Non-Functional Requirements-----	9
Work Breakdown Structure-----	10
Risk Register-----	10
Solution Conceptual Design-----	12
Architecture and Component Design-----	18
Data flow-----	20
Tech Selection-----	20
Design Justification-----	21
Component Level-----	21
NFR-Driven Justification-----	21
Constraints-----	21
Progress status report-----	21
Milestone Achievement Review-----	22
Feedback Integration-----	23
AI Use Disclosure-----	24
Human Oversight-----	25
Contributions-----	25

Group 1 Contract

This contract defines how our group will work together throughout CPRO 2901.
By signing, all members agree to the expectations below.

1. Group Information

Project Title: Paper Party

Group Members:

- | | |
|-----------------------|-----------------------|
| 1. Name: Corbin Lambe | Student ID: 000381573 |
| 2. Name: Isaac Keith | Student ID: 000370277 |
| 3. Name: Shawn Harke | Student ID: 000382668 |
| 4. Name: Harsh LNU | Student ID: 000377703 |

Primary Contact Person:

Name: Isaac Keith, Email: omegawaffleking@gmail.com

2. Group Purpose

The group's goal is to create a legal accounting app that can be used to assist accountants who work at KCF services.

3. Individual Responsibilities

Each member agrees to actively contribute.

Initial role assignments may change as the project evolves.

Group member	Primary Responsibility
Corbin Lambe	Researching accounting procedures as well as assisting in defining requirements and architecture design.
Isaac Keith	Defining applications requirments
Shawn Harke	Creating Architecture
Harsh LNU	Creating Conceptual Design

4. Communication Expectations

- Primary communication method:
iMessage
 - Expected response time to group messages:
Within 24 hours
 - Meetings will be scheduled:
As needed
-

5. Work Standards & Accountability

We agree to:

- Attend scheduled meetings or notify the group in advance
- Complete assigned tasks on time
- Communicate early if difficulties arise
- Treat all group members with respect

Missed responsibilities must be addressed early, not at the end of the term.

6. Conflict Resolution

If issues arise:

1. The group will discuss the issue internally first
2. If unresolved, the group will document the concern
3. The instructor will be consulted if needed

The goal is resolution, not blame.

7. Academic Integrity

All group members agree that:

- Submitted work represents honest effort
 - Contributions are fairly represented
 - Academic integrity policies apply to all project work
-

8. Agreement

By signing below, each member agrees to the expectations in this contract.

Name	Signature	Date
Corbin Lambe	Corbin Lambe	01/09/26
Isaac Keith	Isaac Keith	01/09/26
Shawn Harke		

Harsh LNU	Harsh	01/09/26
-----------	-------	----------

Project Statement

1. Problem Statement

Modern accounting applications currently lack some features that accountants can use and lack clean UI's that make it simple for consumers to use. They can also cost vast amounts of money that many people can not afford. The Paper Party project aims to provide accountants with a platform that enables users to add, update, and delete bank accounts on the website. The service will track their invoices and prepare their taxes. The application will also ensure that the accounts are balanced and that no miscalculations have been made. The Paper Party application will allow users to manage accounts easily and with little challenge. Users will be able to manage and track their expenses, as well as their income. Using the application, users will be able to calculate the amount of tax they owe, making their tax return process much easier. Our version of an accounting app will be free to use and intuitive for the user.

2. Proposed Technical Solution

Desktop Application

3. Stakeholder Identification

KCF Services Ltd. will be using this software to help with accounting,

4. High-Level Scope

In-Scope:

- Expense tracking
- Income tracking
- Tax calculator
- Chart of accounts, edit, add, delete
- Reconciliation to ensure the accounts are balanced

Out-of-Scope:

- Auto-calculate/fill to help users when inputting receipts
- Receipt Capture

5. Preliminary Technology Stack

Winforms, C#, MSSQL

Problem Definition and Scope

Problem Statement:

Modern accounting applications are currently not intuitive, lack organizational features, and have problems with keeping data saved, which significantly slows down the accounting process.

Project objective:

The objective for this application is to design a system that will allow KCF services to create accounts for users and have their taxes done in a clean, organized way with an intuitive design.

High-Level Scope:

In-Scope:

- Create, edit, and delete accounts
- Support account types (asset, liability, equity, income, expense)
- Sub-accounts
- Account numbering
- Create customers and sub-customers

Out-of-Scope:

- Auto-calculate/fill to help users when inputting receipts
- Receipt Capture
- Server capabilities
- Auto Suggestions
- Existing accounts are migrated to this application
- Support PDF generation

Assumptions:

- Receipt data is reliable

Stakeholders & Use context

Stakeholder	Role	Interest
KCF Services	Oversees operations and creates accounts for accountants, and is the admin.	KCF wants an application that will help the accounting process, making it faster and simpler.
Accountant	Updating user data and creating accounts for users, as well as reading the users' data.	KCF wants a more organized and intuitive application.
Tax payer	Ensure they provide complete and reliable information to KCF, and will just read their data.	Reliable and easy tax service.

User Context:

Accountants are in a fast-paced environment where clients can stack up quickly. Their accounting system needs to be organized and intuitive to ensure they can complete their work in a fast and organized manner.

Use Context Scenario:

1. Client goes to KCF to drop off receipts
2. The accountant comes and organizes the data into a file
3. Accountant inputs data into the application

4. The system will sort the data
5. The input data will be compared to the bank statements
6. Accountants use the application to balance an account
7. The application will track income and expenses

Constraints:

- Data loss must be avoided
- Must have a large capacity to hold users' information
- The system will have to be integrated into KCF's current workflow

Functional Requirements

Req ID	Description	Priority
FR-101	The System shall allow accounting firms to create new accounts by clicking the new account button	High
FR-102	The System shall allow accounting firms to create sub-accounts (aka an account under a larger account) by clicking the new sub-account button	High
FR-103	The System shall allow Admins to assign roles	High
FR-104	The System shall allow Admins to assign permissions	High
FR-105	The System shall allow Admins to create user accounts	High
FR-106	The System shall allow Admins to edit user accounts	High
FR-107	The System shall allow Admins to delete user accounts	High
FR-108	The System shall automatically create an audit trail	Medium
FR-109	The System shall allow accounting firms to delete bank accounts by clicking a delete button	High
FR-110	The System shall allow accounting firms to edit bank accounts by clicking an edit button	High
FR-111	The System shall allow accounting firms to access bank account balances by double-clicking in the trial balance	High
FR-112	The System shall automatically give account ID numbers that increase incrementally	High
FR-113	The System shall support multiple account types (assets, liabilities, income, expenses)	High
FR-114	The System shall allow accountants to create customers	High
FR-115	The System shall allow accountants to sub-customers (aka a specific store or company under a larger company) when needed	High
FR-116	The System shall create estimates	High
FR-117	The System shall convert estimates to invoices when clicking the convert button	High

FR-118	The System shall create invoices when the convert button is clicked	High
FR-119	The System shall create balance sheets upon the accountant entering the balance sheet section	High
FR-120	The System shall automatically create a profit & loss section within the balance sheets	High
FR-121	The System shall automatically create a sales by customer section within the balance sheets	High
FR-122	The System shall automatically create an A/P aging section within the balance sheets	High
FR-123	The System automatically creates an A/R aging section within balance sheets	High
FR-124	The System shall automatically create an expenses by vendor section within the balance sheets	High
FR-125	The System shall allow accounting firms to convert statements into a printable PDF by clicking the PDF button	Medium
FR-126	The System shall create customer statements	High
FR-127	The System shall automatically have sales tax tracking	Medium
FR-128	The System shall allow accounting firms to enter bills within bank accounts	High
FR-129	The System shall allow accounting firms to enter receipts	High
FR-130	The System shall restrict access to sensitive accounts	High
FR-131	The System shall allow accountants to make accounts	High
FR-132	The System shall require a work email when making a new account	High
FR-133	The System shall send verification emails	High
FR-134	The System shall deny login access until the email is verified	High
FR-135	The System shall force accountants to log in when accessing a file	High
FR-136	The System shall encrypt data in transit with TLS 1.3	High
FR-137	The System shall encrypt data at rest with AES-256	High
FR-138	The System shall use server-side state to help keep track of session data	High
FR-139	The System shall use tokens to help keep track of session data	High
FR-140	The System shall allow accountants to sort accounts, customers, jobs, invoices, and receipts	High
FR-141	The System shall order accounts alphabetically	High
FR-142	The System shall order customers alphabetically	High
FR-143	The System shall order jobs alphabetically	High

FR-144	The System shall order invoices by date primarily, then by number secondarily	High
FR-145	The System shall order receipts by date primarily, then by the order entered secondarily	High
FR-146	The System shall allow accountants to search for accounts by name	High
FR-147	The System shall allow accountants to search for customers by company	High
FR-148	The System shall allow accountants to search for jobs by company	High
FR-149	The System shall allow accountants to search for invoices by company	High
FR-150	The System shall allow accountants to search for receipts by company	High
FR-151	The System shall allow accountants to search for invoices by number	High
FR-152	The System shall allow accountants to filter for account numbers when searching for accounts	High
FR-153	The System shall allow accountants to filter for account types when searching for accounts	High
FR-154	The System shall allow accountants to filter for companies when searching for invoices	High
FR-155	The System shall allow accountants to filter for a range of invoice numbers when searching for invoices	High
FR-156	The System shall allow accountants to filter for a range of invoice dates when searching for invoices	High
FR-157	The System shall allow accountants to filter for companies when searching for receipts	High
FR-158	The System shall allow accountants to filter for a range of receipt numbers when searching for receipts	High
FR-159	The System shall allow accountants to filter for a range of receipt dates when searching for receipts	High
FR-160	The System shall not allow the creation of new accountants without an account name filled out	High
FR-161	The System shall not allow the creation of new accountants without an account name as a string	High
FR-162	The System shall not allow the creation of new accountants without an account type filled out	High
FR-163	The System shall not allow the creation of new accountants without an account type as a string	High
FR-164	The System shall not allow the creation of a new bank account without an account starting balance filled	High
FR-165	The System shall not allow the creation of a new bank account without an account starting balance as a Decimal	High

FR-166	The System shall not allow the creation of new customers without a name filled out	High
FR-167	The System shall not allow the creation of new customers without a name as a string	High
FR-168	The System shall not allow the creation of new jobs without a name filled out	High
FR-169	The System shall not allow the creation of new jobs without a name as a string	High
FR-170	The System shall not allow the creation of new invoices without a name filled out	High
FR-171	The System shall not allow the creation of new invoices without a name as a string	High
FR-172	The System shall not allow the creation of new invoices without a date chosen	High
FR-173	The System shall not allow the creation of new invoices without a date as a datetime	High
FR-174	The System shall not allow the creation of new receipts without a name filled in	High
FR-175	The System shall not allow the creation of new receipts without a name as a string	High
FR-176	The System shall not allow the creation of new receipts without a date chosen	High
FR-177	The System shall not allow the creation of new receipts without a date as a datetime	High
FR-178	The System shall not allow the creation of new receipts without the company's choice	High
FR-179	The System shall not allow the creation of new receipts without the company as a string	High

Non-Functional Requirements

Non-Functional Requirement ID	Description	Priority
NFR-101	The system shall support up to 30 concurrent users without a degradation in response time	High
NFR-102	The System shall generate reports within 10 seconds of the user selecting it	Medium
NFR-103	The System shall have the front-end available 99% of the month, excluding scheduled maintenance	Medium
NFR-104	The System shall have the back-end available 99% of the month, excluding scheduled maintenance	Medium
NFR-105	The System shall have the database available 99% of the month, excluding scheduled maintenance	Medium
NFR-106	The System shall log the user ID, session ID, timestamp, error code, and transaction ID	High

NFR-107	The System shall retain error logs for 90 days	Medium
NFR-108	The System shall store error logs in LogTide cloud using Docker	High
NFR-109	The system shall perform an automated MSSQL backup every 12 hours	High
NFR-110	The system shall use AES-256 to encrypt data at rest	High
NFR-111	The system shall use TLS 1.3 to encrypt data in transit	High
NFR-112	The system shall hold a technical capacity of up to 1 million transactions a month, without degradation in response time.	High

Work Breakdown Structure

1.0 Requirement Finalization (Week 8)

- 1.1 Research International Accounting Laws(2hrs)
- 1.2 Conduct Peer Review of Functional Requirements (4hrs)
- 1.3 Define Quantifiable Performance Metrics (NFRs) (3hrs)
- 1.4 Security Protocol Research

2.0 System Design & Architecture (Weeks 9-10)

- 2.1 Select tech stack
- 2.2 Create User Workflow Flowcharts (8hrs)
- 2.3 Create Architecture Design Draft (5hrs)
- 2.4 Create UI Draft (4hrs)
- 2.5 Create an architecture design chart
- 2.6 Create a Component design chart
- 2.7 Design MS SQL 2025 Database Schema (ERD) (6hrs)

3.0 Technical Validation / Prototyping (Week 11)

- 3.1 Prototype Database "Create/Read/Update/Delete" Logic in C# (6hrs)
- 3.2 Test UI-to-Backend Connectivity (4hrs)

4.0 Proposal Consolidation (Weeks 12-14)

- 4.1 Draft Executive Summary and Problem Justification (5hrs)
- 4.2 Perform Quality Review of Technical Diagrams (4hrs)
- 4.3 Create Technical Presentation(3hrs)
- 4.4 Revise Forms based on feedback(3hrs)
- 4.5 Finalize Consolidated Capstone Proposal Package (10hrs)

Risk Register

Risk ID	Category	Description of risk	Likelihood (1-5)	Impact (1-5)	Score	Mitigation Strategy	Contingency Plan	Risk Owner
R-101	Technical	Scalability Failure	4	5	20	Make the system horizontall	Notify the users that the system is	Harsh LNU

						y scalable so that it can be upgraded when needed	under high Load and halt any new entries	
R-102	Ethical	Sharing financial information	3	5	15	Enforce AES-256 encryption for data at rest and TLS 1.3 encryption for data in transit	Purge the database and implement a mandatory password reset	Corbin Lambe
R-103	Operational	The system is offline during business hours	3	5	15	Use Blue-Green deployment so there is always one server working	Notify Users when the system is down and when they can expect it to be online	Isaac Keith
R-104	Technical	Database connection issues	3	5	15	Perform a technical spike on the database connection before project completion	Revert to a local database for final design validation	Corbin Lambe
R-105	Ethical	Accounts are hacked	2	5	10	Use PBKDF2 hashing for user passwords	Purge the database and implement a mandatory password reset	Corbin Lambe
R-106	Project	Loss of project data	2	5	10	Frequent commits to the GitHub repository	Revert to the latest GitHub repository	Isaac Keith
R-107	Schedule	Features are not completed in time	2	5	10	Break down tasks for easier development	Simplify features that are unable to be completed	Shawn Harke

R-108	Schedule	Underestimating task complexity	5	2	10	Document features thoroughly, break down tasks to the very smallest detail	Have a group meeting to help simplify the task	Shawn Harke
R-109	Technical	DDoS attacks on the system	2	5	10	Implement caching on the database to handle many concurrent requests	Notify Users when the system is down and when they can expect it to be online	Harsh LNU
R-110	Scope	Scope creep: the project's scope grows past its proposal guidelines	4	2	8	Create a clearly defined scope, and have clear communication between group members.	Revert to the plan when a change is suggested.	Shawn Harke

Solution Conceptual Design

1. High-Level System Architecture Diagram

Tier 1 – Presentation Layer (Frontend)

- WinForms UI (C# Desktop Application)
- Login / Authentication Screen
- Account Manager View
- Invoice & Estimate View
- Reports Viewer (Balance Sheet, P&L, A/R, A/P)

Tier 2 – Application Layer (Backend / Business Logic)

- Account Controller (create, edit, delete accounts + sub-accounts)
 - Invoice & Estimate Controller (FR-112 to FR-115)
 - Report Engine (generates balance sheets, customer statements)
 - Auth / RBAC Service (role-based access control – FR-103, FR-110)
 - Audit Trail Logger (FR-104)
 - Receipt & Bill Entry Handler (FR-119, FR-120)
- #### Tier 3 – Data Layer (Storage)
- MS SQL Server 2025 — Accounts DB
 - MS SQL Server 2025 — Transactions DB (receipts, bills, invoices)
 - MS SQL Server 2025 — Users & Roles DB
 - Automated 12-hour SQL Backup (NFR-105)

2. Conceptual Data Flow

Actor	Data INTO System	Data OUT OF System
KCF Admin	Login credentials, user account setup, and role assignments	Confirmation of account creation, audit logs.
Accountant	Client receipts, bills, bank data, account entries	Invoices, estimates, balance sheets, customer statements, reports
Tax Payer / Client	Personal/financial information (via accountant)	View-only access to their tax data and statements
MS SQL Server	Stored records (accounts, transactions, users)	Retrieved and updated data on request
Printer	PDF	Printable PDF statements and reports (FR-116)

3. Solution Logic Summary

Design Justification:

Student Accounting Assistant is based on a 3-Tier Architecture in which the front-end is based on C# WinForms, the back-end is based on C#, and data is stored in MS SQL Server 2025. Monolithic Architecture is used in place of Microservices Architecture as the application is designed for a limited office environment in which a limited number of users will access the application at a time. It makes the application deployment easier and keeps the project within a reasonable timeframe while being perfectly sufficient for the scale of operations of KCF Services.

Entity Framework Core is used as a medium of communication between C# and MS SQL Server 2025. It is more secure and faster as there is no need to write raw SQL queries, which can lead to injection attacks.

Security:

The first non-functional requirement is related to data security, as the system will handle sensitive financial data, tax data, and personal information of clients. If such sensitive data were to fall into the wrong hands, this would be a major breach of client trust, which would violate IFRS and ASPE standards. The system meets NFR-111 by encrypting all communication between the user interface and backend using TLS 1.3, ensuring that credentials and financial information are never sent in plain text. The system satisfies NFR-110 by encrypting sensitive database fields, using AES-256 via Transparent Data Encryption in MS SQL Server 2025. Additionally, Role-Based Access Control implements FR-103 and FR-110 to prevent unauthorized access to features and data, restricting users to only their assigned roles. Finally, to comply with NFR-106, the system logs the user ID, session ID, timestamp, error code, and transaction ID for every relevant transaction, supporting auditability and security monitoring.

Performance and Concurrency:

The 3-Tier Architecture directly fulfills the system's concurrency and security requirements by strictly separating each layer. Specifically, there is no direct communication between the Presentation Layer (WinForms front-end) and the Data Layer; all requests must pass through the Application Layer first. As a result, the business logic layer acts as a controlled gateway, checking and authenticating every request before proceeding to access the database. This architectural separation effectively eliminates unauthorized direct access to the database and significantly minimizes the risk of data compromise in cases where the front-end interface is compromised. Finally, the 3-Tier Architecture's independent tiers enable individual layer upgrades without affecting the other tiers. The system is designed to support up to 30 concurrent users without compromising response time (NFR-101). This is well within the limitations of a monolithic architecture in a single office setting. Report generation, identified as a resource-intensive task, is completely managed by the Report Engine in the Application Layer and is expected to complete in 10 seconds from the time of user request (NFR-102). Microsoft SQL Server 2025 regulates concurrent read and write operations by employing inherent transaction isolation levels, thus preventing data conflicts during concurrent user access and updates of records in the database. Additionally, Entity Framework Core enhances this capacity by employing automatic connection pooling during concurrent usage periods. Finally, the database is designed to support a technical capacity of up to 1 million transactions (NFR-112) in anticipation of KCF Services' future growth.

Scalability and Maintainability:

While the scope is presently limited to the KCF Services deployment, the 3-Tier Architecture is flexible enough to accommodate future scalability. The fact that the system consists of the presentation layer, the business logic layer, and the data layer means that the system can be easily upgraded as needed, with individual components replaced as required. For example, in the event that the system is required to support more users, the system can be easily upgraded by replacing the MS SQL Server with a more powerful server. In addition, the system can support more controllers as well as reports within the application layer without altering the front end. The use of Entity Framework Core also helps the system maintainability in the long run, as the data is defined within the C# code as opposed to raw SQL scripts.

Backup and Recovery:

The integrity of data is ensured through an automated 12-hour SQL backup plan (NFR-109). This will ensure that in case of a hardware failure or data corruption, the system will be able to restore itself to a recent state with minimum loss of data. This will be done at an MS SQL Server level, meaning that it will be done independently of the application and will not impact system performance during business hours.

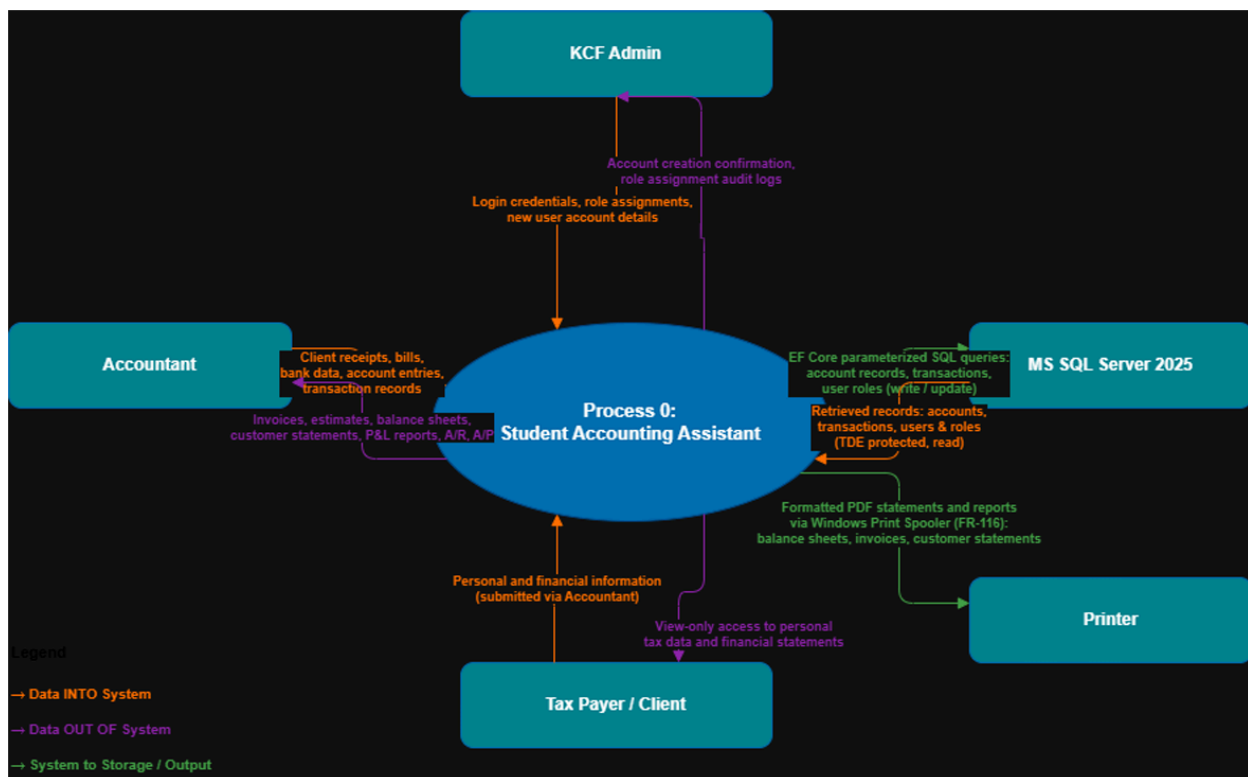


Figure 1: The diagram illustrates all external actors that interact with the system and the data that flows in and out. The KCF Admin manages user roles and accounts, the Accountant inputs and retrieves financial data, the Tax Payer has view-only access, and the system communicates with MS SQL Server for data storage and a printer for document output.

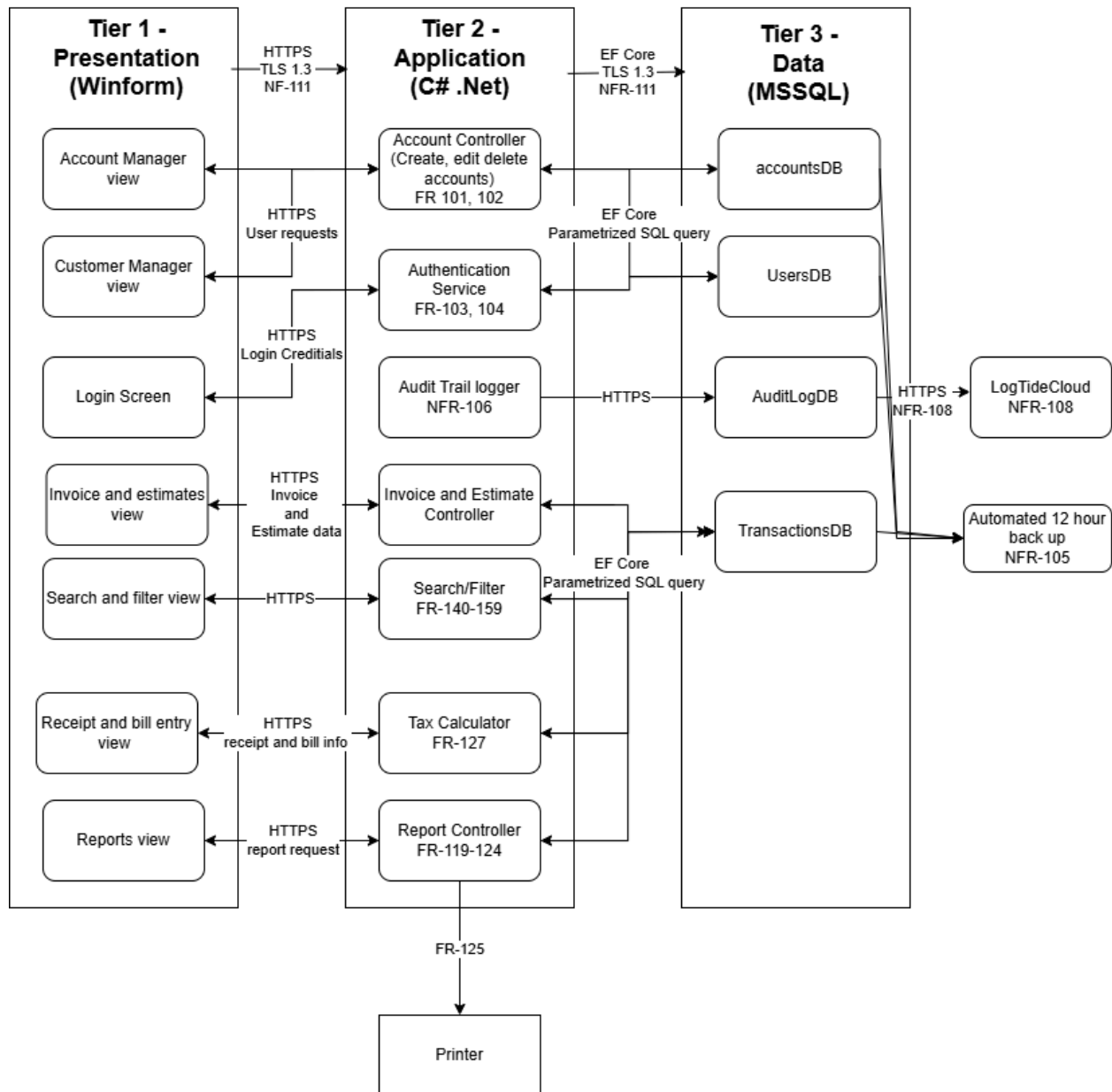


Figure 2: High-level system architecture Diagram

Application Layer: All user interactions from the WinForms UI are sent to the backend through encrypted local channels using HTTPS. This ensures that login information, form data, and data requests are not sent in clear text

Application Layer → Data Layer: The backend only communicates with MS SQL Server 2025 through the Entity Framework Core. The Entity Framework generates SQL queries with parameters. SQL injection is thus not possible. Moreover, these queries are sent over a trusted internal network connection, and data at rest is encrypted using AES-256 to meet NFR-110

Application Layer → Printer: Print requests are sent in the format of PDF files generated by the Report Engine. The PDF files are sent over the local network to the printer through the standard Windows Print Spooler service.

Data Layer → Application Layer: MS SQL Server returns the retrieved data to the business logic layer over the same trusted internal network connection. Sensitive data, such as passwords and financial data, is encrypted at rest through AES-256 to meet NFR-110

4. External Dependencies and Third-Party Systems

Overview:

The Paper Party Application is a fully self-contained desktop application. After a comprehensive analysis of the project requirements and the operational environment of KCF Services, the team has determined that no third-party APIs are required for the operation of this application. The application is fully self-contained and utilizes only internal dependencies for all three tiers of operation.

No Third-Party APIs:

The application does not utilize any third-party APIs for operation. The application's functionality for generating invoices, reports, managing accounts, and handling receipts is fully self-contained and carried out internally within the application layer via custom C# controllers. The application does not utilize any third-party invoicing systems, payment gateways, or financial data providers. This decision was made to ensure a fully operational application without any external dependencies and to maintain a fully operational application within a closed office network without the need for internet access.

Cloud Services:

Paper party will be using LogTide cloud due to its easy integration, low budget, and Logging capabilities. It will be run on KCF servers, giving the company full control over data and increasing financial data security. The service also has end-to-end tracking. Log tide can detect suspicious behaviours like someone trying to log in multiple times.

No Licensed Third-Party Libraries Beyond Standard Stack:

The third-party libraries and tools utilized in this project are industry-standard and free/open source:

- C# / .NET – Application Development Framework (Microsoft)
- Entity Framework Core – Object-relational Mapping Framework for Database Access (Microsoft)
- MS SQL Server 2025 – Relational Database Management System (Microsoft)
- Windows Print Spooler – Native Windows Service for Print Output (Microsoft)

None of these libraries and tools requires a third-party license agreement in addition to standard Microsoft Development Tools licenses; all should already be included in the standard technology stack for a Windows Desktop Application of this nature.

Impact on System Design:

The lack of third-party libraries has several direct advantages:

Reliability – The application is not subject to outages of third-party systems, API deprecation, or service updates outside of the project team's control.

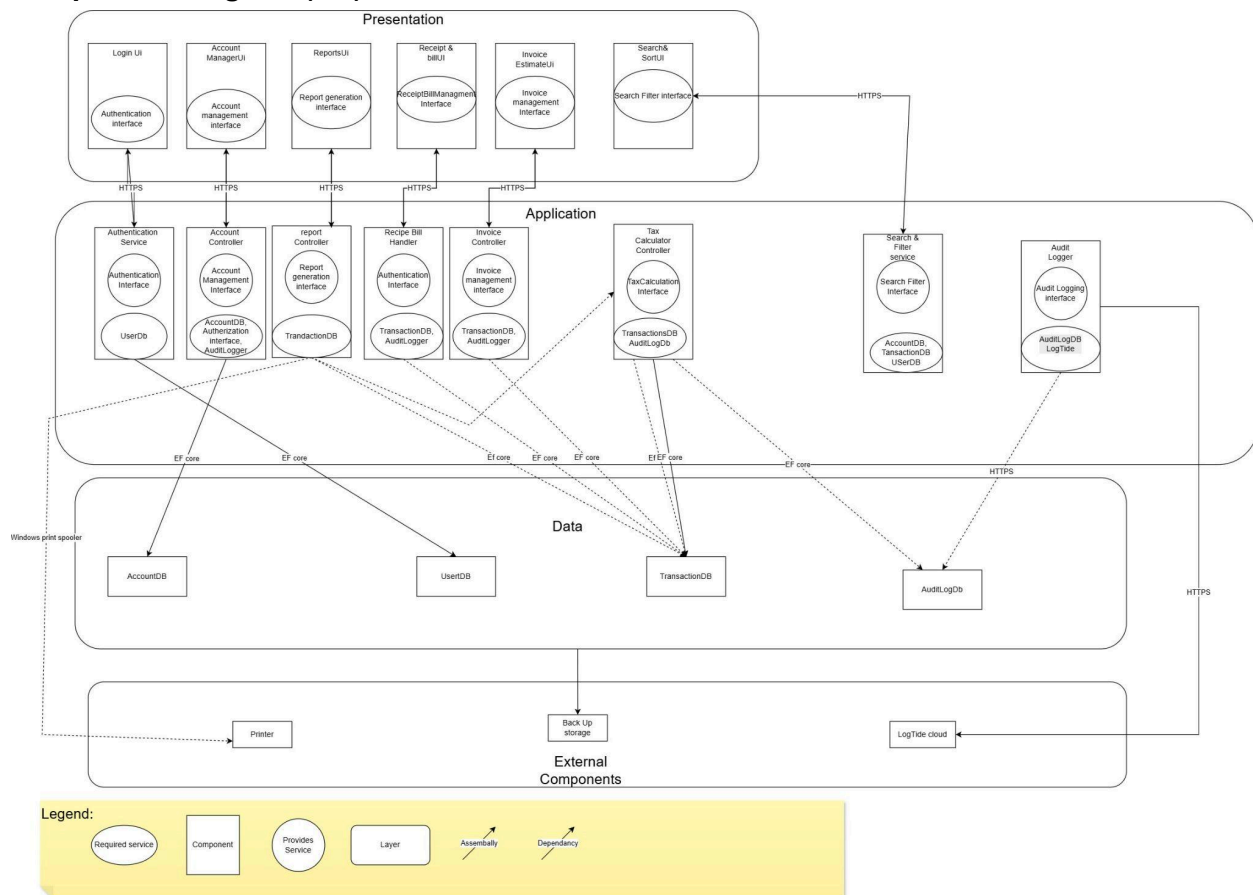
Security – There are no third-party systems into which data could potentially be intercepted and leaked.

Ease of deployment – The system can be deployed directly onto a machine or office server without needing access to the Internet or a third-party system.

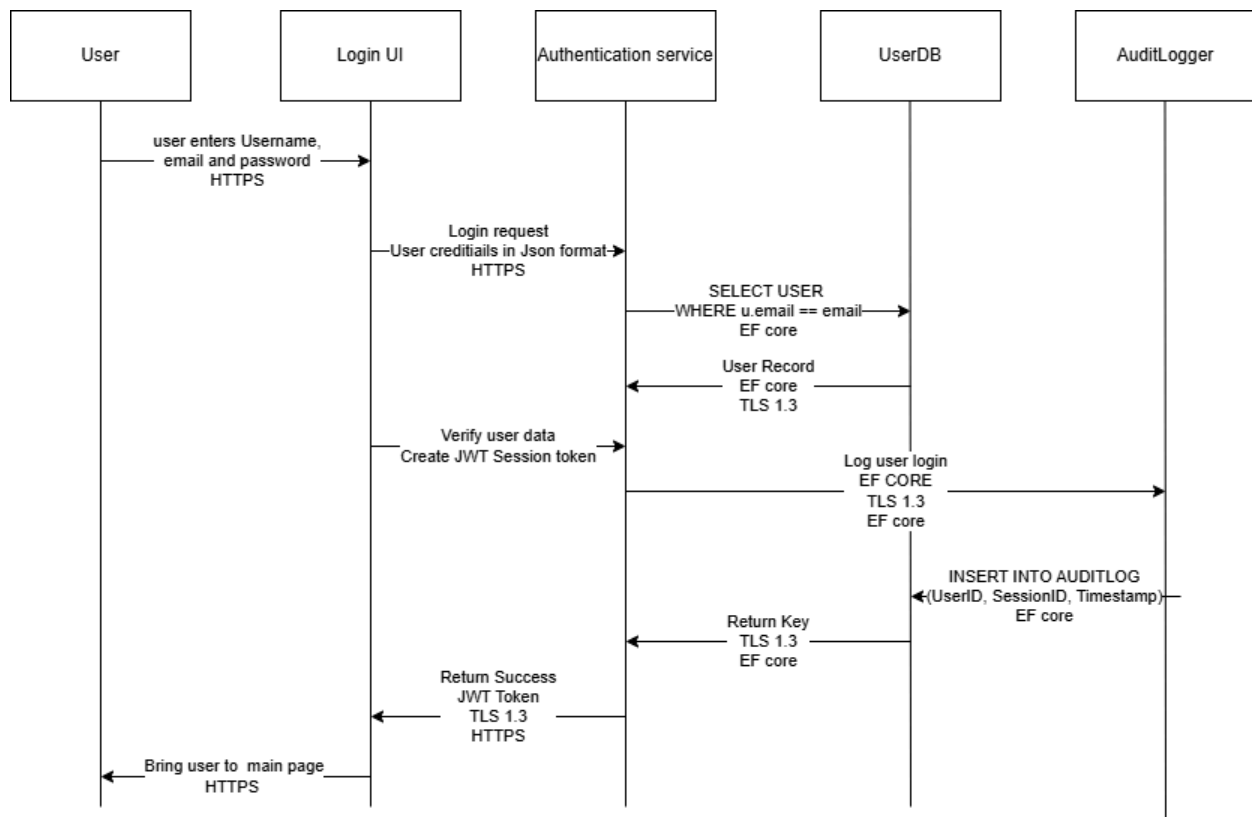
Ease of maintenance – There are no third-party system updates and/or subscriptions to monitor and update in future iterations.

Architecture and Component Design

Component Diagram (2.2):



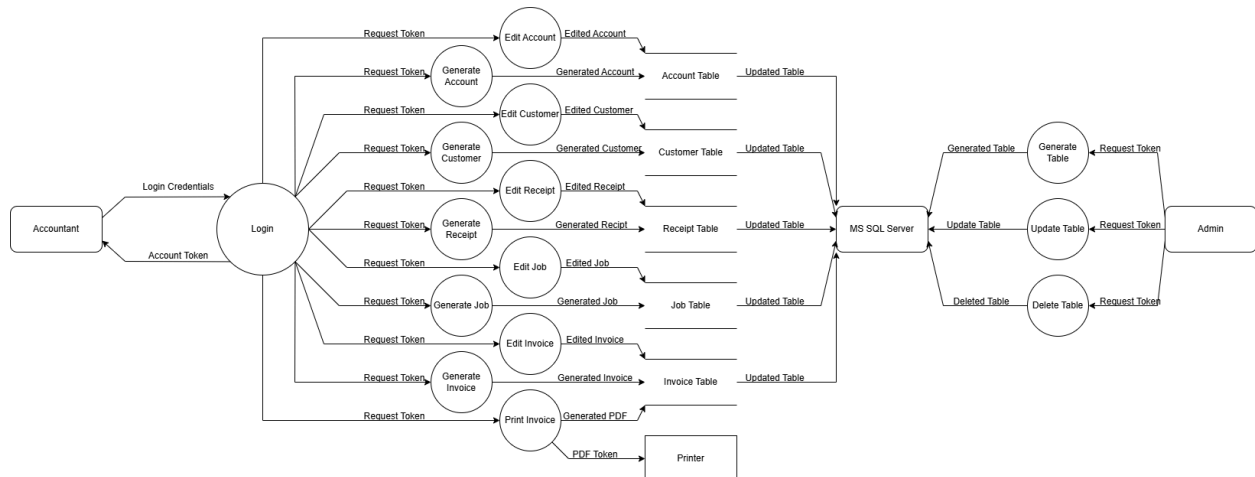
Interface & API Specification (2.2):



Technology Stack Justification:

Layer	Chosen Technology	Technical Rational
Frontend	C# WinForms	C# WinForms was chosen for three reasons. Firstly, our program needs to be able to function offline so the accountants can keep working even when their internet becomes buggy. Secondly, WinForms ' simplicity allows for quick re-drafts when components don't end up being satisfactory for our client. Lastly, there is the compatibility and stability with our backend C#, allowing for the application to run seamlessly and predictably.
Backend	C#	C# was selected due to its being strongly typed, which is needed to prevent run-time errors and to keep data consistent. Furthermore, as the application needs to support several users simultaneously, async patterns are needed to support thirty concurrent users. As the application will be actively engaged with client-sensitive information, we need its role-based security features and JIT compilation.
Database	MSSQL	MSSQL was chosen because it has row-level locking, which will enable the system to have concurrent operations for the needed number of users. MSSQL has native support for TLS 1.3 encryption, which will be used to encrypt data in transit. Furthermore, it also has native support for AES-256 encryption, which will be used to encrypt data at rest, which is heavily needed as the system will be dealing with high amounts of sensitive user data.

Data flow



Tech Selection

Layer	Chosen Technology	Technical Rational
Presentation	C# Windows Forms	Selected due to its drag-and-drop feature, which will accelerate the development time
Backend	C#	Selected due to its being strongly typed, which can prevent run-time errors and keep data consistent. Async patterns will support thirty concurrent users. It also has role-based security features and JIT compilation for better performance.
Database	MSSQL	MSSQL has row-level locking, which will enable the system to have concurrent operations. It also has native support for AES-256 encryption, which will be used to encrypt data at rest, and it also has native support for TLS 1.3 encryption. This will be used to encrypt data in transit.
Framework	.NET 8 Framework	Allows for consistent development across presentation, back-end, and authentication layers. The .NET framework also has improved encryption and identity tools to help the system meet accounting security standards.
Authentication	ASP.NET Core Identity	Selected due to its stateless authentication, which will be required for horizontal scaling, PBKDF2 password hashing will help the application meet security requirements, and it has integrations with MSSQL via Entity Framework Core.
Cloud Hosting	LogTide Cloud	LogTide Data will be held on user servers, which will give the user full data ownership and privacy control, which will help the system follow Accounting security practices, and it has built-in audit logs that record the user, what data was accessed, when it was accessed, and where it was accessed.

Design Justification

The Paper party accounting project will be using a three-tier architecture. We chose a three-tier architecture because it provides separation between presentation, business logic, and data storage. This allows us to keep the database hidden from users, making the platform more secure. Having this system also allows us to scale each tier independently and work on tiers in parallel.

Presentation
Application
Database

Component Level

The Accounting assist engine manages state transitions and enforces business logic. It also controls how sub-components interact. The accounting assist engine will rely on abstraction to prevent the components from being tightly coupled. The system promotes maintainability because if new regulations are put in place, only the Accounting assist logic will need to be modified; the tax calculator and invoice manager logic will stay the same.

NFR-Driven Justification

The system shall hold a technical capacity of up to 1 million transactions a month, without degradation in response time. A three-tier architecture will allow the system to separate tiers, to scale independently, and isolate risks.

Constraints

Some constraints that impacted our choice of design were our skill set; we all have differing strengths, and a three-tier architecture allows us to each work separately on tiers that reflect our skills. Having a three-tier architecture also allows us to test tiers separately, which will speed up the development process, allowing us to develop the application within the next 6 months without having to wait for each other. The architecture will also allow us to scale horizontally to match the non-functional requirements of the system, holding 1 million transactions a month and having up to 30 concurrent users.

Progress status report

Completed work:

The Paper Party accounting assist project has clearly defined non-functional and functional requirements that have been adjusted to match the feedback that was given. Data flow, High-end architecture, and component design charts have been created. The technology that will be used in the development of the accounting system has been selected and confirmed to follow Canadian accounting laws as well as international accounting practices.

Work in progress:

The Architecture and Component design diagrams are being revised to follow the feedback given. The database has been created, but proper schemas have not been clearly defined, and still need to follow Accounting security practices. Research on Accounting practices will also continue, so we can be sure our application is following international accounting laws. The Design justification is still being worked on, and LogTide is still being researched.

Upcoming work:

The Angry Toasters team currently needs to do the following:

- Final Consolidation Submission
- Create a rough draft of the application
- Load test application
- Consult KCF services with the project documentation.

Risks, Issues, and Mitigation:

The project has two major risks: team coordination and missing components. Some of our team members are hard to get a hold of and have projects due in other classes, which results in a limited number of team members contributing to Upcoming work and revisions that need to be made. Our project may be missing components in our Component diagram due to a lack of complete understanding of what will be needed for our project. We plan on mitigating these risks by researching what components are currently being used in existing accounting services, and by having daily communications on the progress being made, and creating mandatory weekly meetings.

Milestone Achievement Review

Milestone	Planned Date	Actual progress	Status
Project Concept Defined	1/22/2026	Complete	Completed
FR Completed	3/5/2026	Complete	Completed
NFR Completed	3/5/2026	Complete	Completed
Conceptual Design Completed	3/19/2026	Complete	Completed
Architecture & Component Design Completed	4/10/2026	Complete	Complete
Data Flow Developed	3/31/2026	A draft has been made and awaits review from team members to ensure the diagram is correct	Partial
Tech Selection Completed	3/31/2026	Complete	Completed
WBS Created	3/5/2026	Complete	Completed

Risk Register Completed	3/12/2026	Complete	Completed
Revision based on feedback	3/30/2026	Complete	Completed
Current draft of Final Proposal	4/10/2026	Complete	Complete
Draft of Application	5/10/2026	The database has started to form	Partial

Achievements

The following Key Milestones have been completed and submitted: Project Concept, FR, NFR, Conceptual Design, Tech Selection, WBS, Risk register with mitigation plans. Every assignment before 03/31/2026 has been handed in before the due date. The NFRs have been revised to include specific security protocols, provide a quantifiable metric for error handling, and specify availability times for the application to match the feedback provided. The FRs have been revised to include user authentication, login credentials, and session behavior. It was also revised to have specific search and sort functionalities for accounts. We also made the measurables more concise, and functions were made atomic.

Milestones Behind Schedule:

Currently, revisions for the conceptual design, as well as the components and architecture design, have not been made due to scheduling conflicts and communication issues. The progress on these assignments remains unknown. Due to the designs not being finalized yet, the Design justification assignment is being affected. Progress has been made on the Design Justification, but it can not be completed until the other assignments are revised.

Catch-Up Plan:

The team plans on finishing these assignments by 03/31/26 by having an in-person group meeting and ensuring these assignments are revised to match the feedback. Shawn Harke has been assigned the responsibility for completing the Architecture and Component design as well as finishing his part of the Components justification assignment. Harsh LNU is responsible for completing the Conceptual Design. The angry toaster team would appreciate feedback on assignments and more assignments to ensure the assignments are up to the course's standard.

Feedback Integration

Phase 1:

- Revised risks to be more specific

Functional Requirements:

- Divided the requirements into one action each
- Added necessary categories that were missed
- Revises descriptions to provide greater detail
- Fixed formatting issues

Non-Functional Requirements:

- Specified security protocols
- Made error handling measurable
- Added more performance requirements
- Revises descriptions to provide greater detail
- Clarified scalability requirements

Solution Conceptual Design:

- Added protocols to the architecture diagram
- Added DFD data labels
- Made the securities justification more explicit
- Added dependencies

Architecture & Component Design:

- Revised component diagram to include internals and how they connect
- Fixed the sequence diagram to show technical protocols and represent an actual loop
- Adjusted justification to match constraints
- Added labels to direct to WBS
- Remade charts in the requested editor

AI Use Disclosure

Component / Deliverable	AI Tool Used	How AI Was Used	Human Contribution %
Title Page	N/A	N/A	100%
Table of Contents	N/A	N/A	100%
Group Contract	N/A	N/A	100%
Problem Concept Statement	N/A	N/A	100%
Problem Definition & Scope	N/A	N/A	100%
Stakeholder & Use Context	N/A	N/A	100%
FR	N/A	N/A	100%
NFR	N/A	N/A	100%
Risk Register	DeepSeek	Used to aid in the research of accounting laws for the initial draft	90%

Conceptual Design	N/A	N/A	100%
Architecture / Component Design	Chat GPT	Used to aid in the understanding of instructor feedback	90%
Data / Process Flow	N/A	N/A	100%
Tech Selection	N/A	N/A	100%
Design Justification	N/A	N/A	100%
Progress Report	N/A	N/A	100%
Milestone Review	N/A	N/A	100%
Feedback Integration Summary	N/A	N/A	100%
Appendices	N/A	N/A	100%

“We self-declare that the overall AI contribution to this final proposal is approximately 1.2%, and our team contributed 98% through our own work, decisions, writing, and technical development.”

Human Oversight

Overall, in this project, we barely used AI. For our project, we wrote everything manually, only using AI to help our understanding of procedures and terms for the accounting field and to research accounting laws. We can assure that most of our members understand the included material.

Contributions

KCF services were consulted to learn about the Functional Requirements and the Non-functional Requirements they would like the app to meet.